Design and Development of an Autonomous Mobile Robot with PID-Based Line Following and Obstacle Avoidance

Xintong Feng, Shamanta Islam, Paula Cho, Elvis-Ledoux

Systems Engineering Project
Group E2–E4
Hochschule Hamm-Lippstadt (HSHL)

Abstract— This project presents the design and development of an autonomous mobile robot capable of line following and obstacle avoidance. The system integrates an Arduino Uno, infrared sensors, ultrasonic sensors, DC motors, and PID-based control to achieve reliable navigation on a predefined track. The robot was designed, simulated, assembled, and tested to evaluate sensor performance, motor control, and overall system reliability. Experimental results demonstrate successful line tracking, obstacle detection, and autonomous manoeuvring, while highlighting opportunities for future improvement in adaptive obstacle avoidance.

Keywords—Autonomous mobile robot, line following, obstacle avoidance, PID control, infrared sensors, ultrasonic sensors, Arduino Uno.

1. Introduction and Objectives

1.1 Background

Autonomous vehicles have become an important topic in robotics because they can perform tasks with little or no human intervention. One common example is a line-following robot, which uses sensors to detect and follow a predefined path. Such robots are widely used in warehouses, factories, and automated production systems to transport materials efficiently while reducing manual work. Because of their practical applications, line-following robots are often used in educational projects to study embedded systems, sensor integration, and motor control.

1.2 Problem Statement

Line-following robots often struggle to maintain accurate tracking, especially during sharp turns, under inconsistent lighting conditions, or when unexpected obstacles appear on the path. This project addresses the challenge of designing a small autonomous vehicle that can reliably follow a black line while detecting obstacles, performing avoidance manoeuvres, and returning to the original track.

1.3 Project Goal

The goal of this project is to design and build an autonomous mobile robot capable of navigating a predefined track while demonstrating basic autonomous behaviour through line following and obstacle avoidance.

The specific objectives of this project are:

- Design and construct an autonomous vehicle.
- Follow a predefined black line.
- Detect obstacles.
- Perform an obstacle-avoidance manoeuvre and safely return to the original track.
- Control the DC motors.
- Integrate all hardware and software components into one complete system.
- Test and evaluate the prototype under different operating conditions.

1.4 Project Constraints

- The robot must be built using low-cost components, including an Arduino Uno R3, IR sensors, ultrasonic sensors, and simple DC motors.
- The system operates on battery power, which limits runtime and requires efficient motor control.
- IR and ultrasonic sensors have limited range and accuracy, especially under varying lighting and surface conditions.
- The robot must navigate a predefined track containing curves and obstacles using PWM and PID-based control.
- Development and testing must be completed within the course timeframe.

2. System Design and Engineering Approach

2.1 Design Approach

The system was designed using a top-down modular approach. We first defined the overall system architecture and then decomposed it into four major subsystems. Each subsystem was assigned a specific functional responsibility to ensure a clear structure, straightforward integration, and reliable operation.

2.2 System Architecture Overview

The autonomous robot consists of four interconnected subsystems:

- Power Subsystem
- Sensor Subsystem
- Control Subsystem
- Motor Subsystem

The functional requirements of each subsystem are listed below.

- **Power Subsystem:** Supplies stable voltage and current.
- **Sensor Subsystem:** Detects environmental changes and generates measurement signals.
- **Control Subsystem:** Processes sensor measurements and produces control commands.
- **Motor Subsystem:** Converts control commands into mechanical motion.

Together, these subsystems fulfil the functional requirements of the autonomous robot, with each subsystem performing a specific role in the overall operation of the system.

2.3 System Data Flow

The autonomous robot operates through a continuous flow of electrical signals and power between the four subsystems.

- **Sensor Subsystem → Control Subsystem**
  - IR sensors send digital signals indicating whether the robot is on the line.
  - Ultrasonic sensors send distance measurements for obstacle detection.
- **Control Subsystem → Motor Subsystem**
  - The Arduino processes sensor inputs and determines whether the robot should move forward, turn, stop, or

avoid obstacles.
  - PWM signals and direction commands are generated for the motor driver.
- **Motor Subsystem → Mechanical Motion**
  - The motor driver amplifies the control signals and drives the DC motors.
  - The motors perform the required movements (forward, left, right, and stop).
- **Power Subsystem → All Subsystems**
  - The battery supplies stable voltage to the Arduino, sensors, motor driver, and motors.
  - Continuous power enables sensing, control, and actuation.

This data flow enables all subsystems to operate together, allowing the robot to achieve reliable line following and obstacle avoidance.

3. System and UML Diagrams

To support the development of the autonomous vehicle, a set of system and UML diagrams was created to model the architecture, behaviour, and interactions within the system. These diagrams provide a structured understanding of how the robot operates, how its subsystems communicate, and how the control logic responds to sensor inputs.

3.1 Block Definition Diagram

Fig. 1 shows the high-level system components and their relationships.It divides the system into four main subsystems: the power supply system, sensing subsystem, control system, and driving subsystem. Each subsystem contains the main hardware components used in the project. The power supply subsystem includes the battery and power module. The sensing subsystem contains the IR sensors and ultrasonic sensors. The control subsystem is represented by the Arduino Uno. The driving subsystem includes the motor driver and DC motors. In addition, the wheel assembly is shown as a separate part of the system because it is an independent mechanical assembly rather than a component of the motor subsystem. This diagram provides a clear overview of the hardware architecture before the detailed diagrams are introduced.

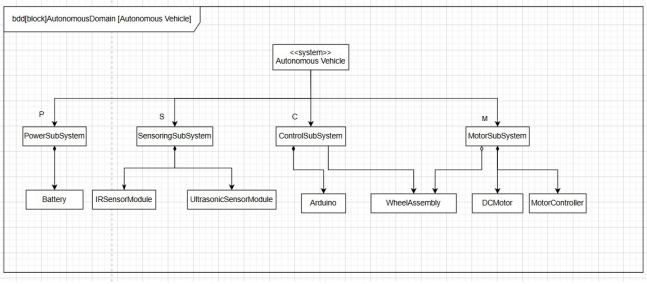

Fig. 1. Block Definition Diagram

3.2 Internal Block Diagram

Fig. 2 presents the Internal Block Diagram of the robot system. Unlike the Block Definition Diagram, this diagram focuses on the internal connections between the main subsystems and hardware components. The power subsystem supplies power to the control subsystem, which is built around the Arduino Uno. The sensing subsystem contains the IR sensor and the ultrasonic sensor, both of which are connected to the Arduino through defined interfaces. Based on the received sensor information, the Arduino generates motor commands and sends them to the motor subsystem. The motor subsystem consists of the motor controller and the DC motor, which are responsible for driving the robot. By showing these connections, the diagram provides a clearer understanding of how information and control signals are exchanged between the different subsystems.

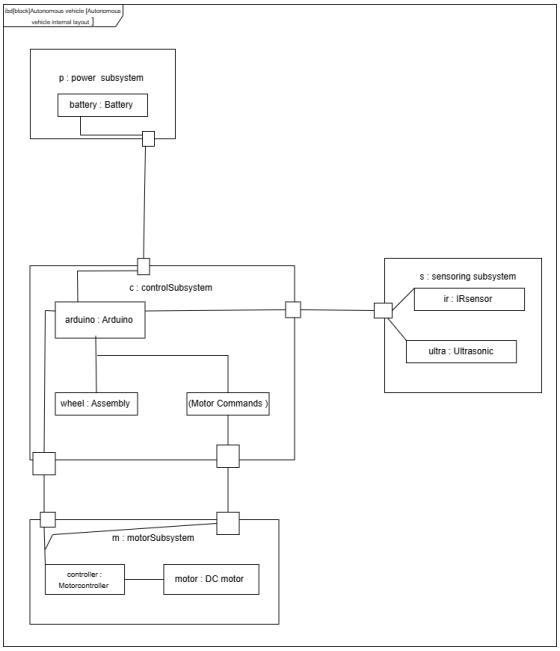

Fig. 2. Internal Block Definition Diagram

3.3 Activity Diagram

Fig. 3 shows the swimlane-based workflow of the autonomous vehicle. It is divided into four swimlanes: IR sensors, ultrasonic sensors, controller, and drive motors.

The swimlanes show which system component performs each activity, while the decision nodes show how the controller selects PID line following, obstacle bypass, or a 180-degree U-turn. The arrows show the order of the activities and the flow between the four system components.

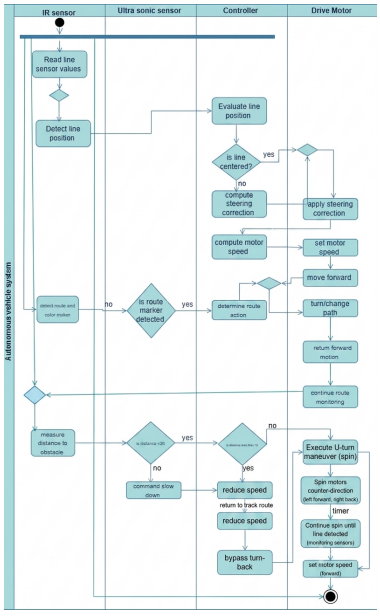

Fig. 3. Activity Diagram

3.4 Sequence Diagram

Fig. 4 shows the sequence diagram of the autonomous vehicle, including five lifelines: the IR sensors, Ultrasonic sensors, Arduino, motor driver, and DC motors.

The messages are arranged from top to bottom in time order. The `alt` frame shows three cases: normal line following, first-obstacle bypass, and second-obstacle 180-degree U-turn. The `loop` frame shows that the process repeats continuously.

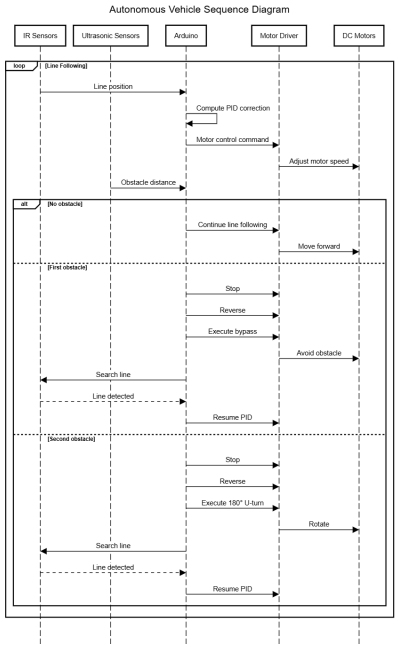

Fig. 4. Sequence Diagram

3.5 Use Case Diagram

Fig. 5 shows the main functions of the autonomous vehicle system and the conditions under which different actions are performed. It illustrates how the robot supports line following, route navigation, and obstacle management through a set of related functions. These functions include line detection, colour identification, automatic steering, distance measurement, obstacle avoidance, stopping, and performing a U-turn. The *include* relationships indicate functions that are always executed as part of a larger task, while the *extend* relationships describe optional actions that are triggered only under specific conditions. The condition notes further explain when the robot adjusts its movement, avoids an obstacle, or performs a U-turn. This diagram provides a clear overview of the robot's functional behaviour before the detailed implementation is introduced.

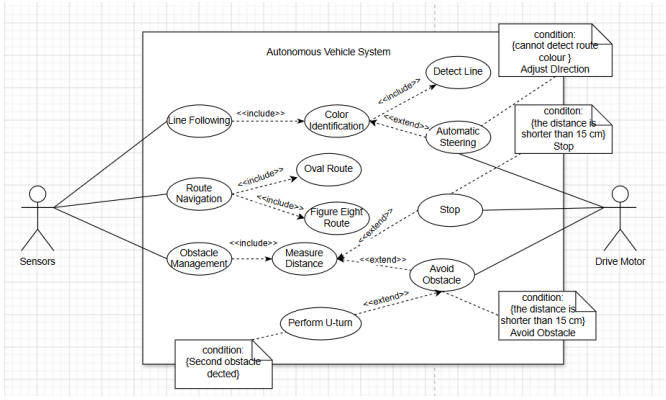

Fig. 5. Use Case Diagram

4. Risk and Mitigation

Table 1 summarises the potential risks encountered during the project and the actions taken to minimise these risks.

Risk Description	Mitigation Strategy
IR Sensor Misalignment or Inaccurate Line Detection	• Calibrate the IR sensors. • Adjust the sensor sensitivity.
Low Battery Voltage Affecting Motor Performance	• Check the battery before testing. • Replace it if necessary.
Hardware Damage	• Check all wires and components. • Secure all parts before testing.
Software or PID Problems	• Test each function separately. • Fine-tune the PID values. • Back up the code using GitHub.
Obstacle Avoidance Failure	• Test the ultrasonic sensors. • Adjust the delay and timeout values.

Table 1. Project Risks and Mitigation Strategies

5. Hardware and Software Implementation

5.1 Hardware Components and Connections

5.1.1 Components

Table 2 lists the hardware components used in the autonomous vehicle. It presents the quantity and purpose of each component required for the assembly.

Component	Quantity	Purpose
Arduino Uno R3	1	Main controller.
L298N Motor Driver	1	Motor control.
DC Motor	2	Drive the wheels.
Infrared Reflective Sensor	2	Detect the black line for line following.
Ultrasonic Sensor HC-SR04	2	Obstacle detection.
Battery	1	Power supply.
Chassis	1	Robot frame.
Wheel	2	Robot movement.
Push On/Off Switch	1	Turn the robot on and off.
Breadboard	1	Connect and organize the electronic components.

Table 2. Components Used for Assembly

5.1.2 Connections

Figure. 6 illustrates the wiring connections of the autonomous vehicle. It shows how the Arduino Uno is connected to the motor driver, sensors, battery, and other hardware components together with their pin assignments.

The motor speed is controlled through the PWM inputs (ENA and ENB) of the L298N motor driver [1].The Arduino Uno provides six PWM-capable digital pins (D3, D5, D6, D9, D10, and D11), which are generated by three hardware timers. Timer0 controls pins D5 and D6, Timer1 controls pins D9 and D10, and Timer2 controls pins D3 and D11 [2].

In this project, the PWM inputs of the motor driver were connected to pins D5 and D6 because both pins are controlled by Timer0. This allowed both motor channels to share the same timer configuration, making it possible to adjust the PWM frequency of both motors by configuring only one timer. Otherwise, if ENA and ENB were connected to PWM pins

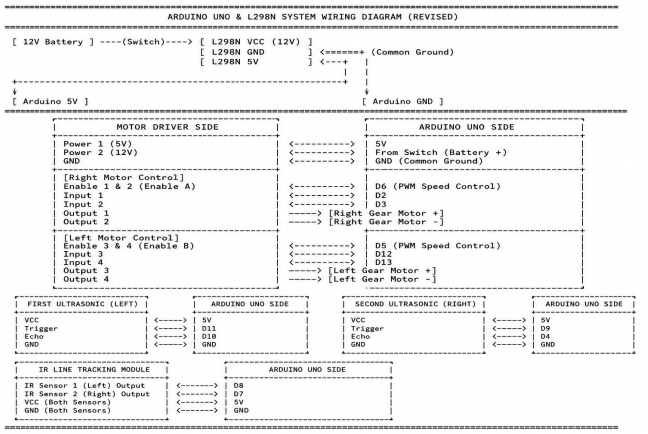

Fig. 6. Autonomous Vehicle Wiring Diagram

controlled by different timers, each timer would need to be configured separately.

The Timer0 prescaler was modified in software to increase the PWM frequency, resulting in smoother motor operation and reduced audible motor noise. The motor speed was then adjusted independently by changing the PWM duty cycle, allowing accurate speed control during PID-based line following.

```
TCCR0B = TCCR0B & B11111000 | B00000010;
```

5.1.3 Mechanical Design

The holders were precisely designed and dimensioned using SolidWorks to ensure accurate fitting and stable mounting of each component.

Figure. 7 illustrates the placement of the 3D-printed holders on the robot chassis. The motor holders, ultrasonic sensor holders, IR sensor holder, and battery/breadboard holder are positioned to securely mount each component. The two IR sensors are mounted at the centre of the front of the chassis to improve black line detection and provide balanced feedback for accurate line-following.

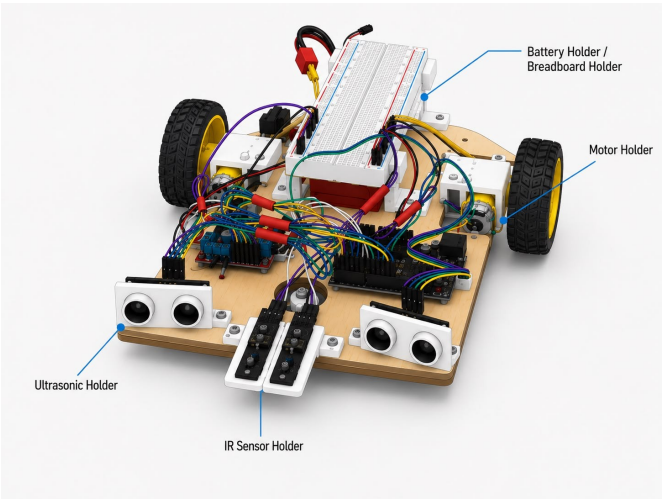

Fig. 7. Holder Placement on the Robot Chassis

6. Software Design

6.1 Control Logic

Fig. 8 shows the main control logic of the robot. Under normal conditions, the robot follows the black line using two IR sensors and a PID controller. When an obstacle is detected, the program checks how many obstacles have already been encountered. For the first obstacle, the robot performs a bypass manoeuvre. When the second obstacle is detected, the robot makes a 180° U-turn. After the obstacle-handling process is completed, the robot searches for the black line again and continues line following.

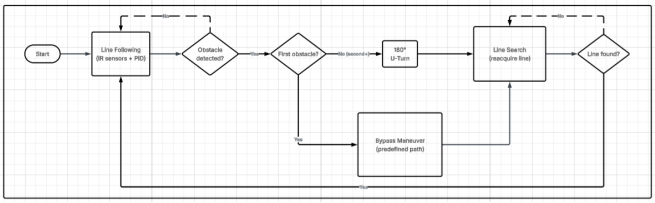

Fig. 8. Software Control State Diagram

6.2 Code Architecture

The software was divided into several functional modules, as summarised in Table 3. The `setup()` function initialises the sensors, motor driver, PWM configuration, and serial communication. The `loop()` function acts as the main control routine and continuously checks the sensor readings to determine the robot's next action.

During normal operation, the `pidLineFollow()` function controls line following using the two IR sensors and the PID controller. When an obstacle is detected, the program selects either the first-obstacle bypass or the second-obstacle U-turn according to the value of the obstacle counter.

Module	Main Function(s)	Purpose
System Initialization	`setup()`	Initialises the IR sensors, ultrasonic sensors, motor-driver pins, PWM settings, and serial communication. It also stops both motors before the robot starts moving.
Sensor Processing	`getDistance()` `readSensor Filtered()`	Measures the distance between the robot and an obstacle using the ultrasonic sensors. Measures the distance between the robot and an obstacle using the ultrasonic sensors. It also reads the digital signals from the two IR sensors and helps reduce unstable readings caused by shadows and reflections.
Decision Making (Main Control Logic)	`loop()`	Continuously checks the sensor readings and selects one of three operating modes: normal PID line following, first-obstacle bypass, or second-obstacle 180-degree U-turn.
PID Line-Following Control	`pidLineFollow()` `resetPID()`	Calculates the PID correction and adjusts the left and right motor speeds to keep the robot on the black line. The PID values are reset after an obstacle-handling manoeuvre.
First-Obstacle Bypass	`avoidTurnLeft()` `avoidTurnRight()` `setMotors()`	Executes the bypass sequence for the first obstacle. The robot reverses, turns away from the line, moves around the obstacle, and turns back until the black line is detected again.
Second-Obstacle U-Turn	`spinRight()` `setMotors()`	Reverses the robot slightly and performs a 180-degree in-place rotation. It continues rotating until one of the IR sensors detects the black line again.
Motor Control	`setMotors()` `stopMotors()` `avoidTurnLeft()` `avoidTurnRight()` `spinRight()`	Controls the direction and PWM speed of both motors. These functions provide forward motion, reverse motion, curved turning, in-place rotation, and stopping.

Table 3. Software Modules and Main Functions

6.3 PID Controller

To achieve smooth and accurate line following, a PID (Proportional–Integral–Derivative) controller was implemented.

The controller adjusts the speeds of the left and right motors according to the line-tracking error detected by the infrared sensors. The control output is calculated as follows [3],p. 314:

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt}$$

where $u(t)$ represents the control output, $e(t)$ is the tracking error, and K_p, K_i, and K_d are the proportional, integral, and derivative gains, respectively. The proportional term reacts to the current error, the integral term reduces accumulated errors, and the derivative term predicts future changes in the error, resulting in smoother steering and improved line-following performance.

6.4 Obstacle Avoidance Software Implementation

The obstacle avoidance program was designed step by step. During the first tests, the robot could only complete the first movement and did not continue with the following movements. Therefore, the complete obstacle avoidance sequence did not work correctly.

To solve this problem, separate movement functions were created based on the `setMotors()` function. These functions control the direction and speed of the two motors. This made each movement easier to test and adjust.

For the first obstacle, the robot first moves backward. Then it turns left to leave the black line. After that, it moves forward beside the obstacle. Finally, it turns right to find the black line again.

A very important improvement was the use of an `if` condition after the first turn. The condition

```
LEFT_SENSOR == 0 && RIGHT_SENSOR == 0
```

means that both IR sensors detect the white surface. This shows that the robot has left the original black line. When this condition is true, the program starts the forward movement beside the obstacle.

This condition was very important during the tests. Without this sensor check, the robot could complete the first movement, but the following movement sequence did not work correctly. After adding the condition, the program could check that the robot was in the correct position before starting the next movement.

After passing the obstacle, the robot starts turning right towards the black line. A `while` loop checks the right IR sensor until the black line is detected again. The turning command is sent before the loop. The motor driver keeps the last direction and PWM values, so the robot continues turning while the program checks the sensor. A timeout is also used to prevent the robot from remaining in the loop for too long.

For the second obstacle, the `spinRight()` function is used to make a 180-degree U-turn. The robot first moves backward for a short distance and then rotates until one of the IR sensors detects the black line again.

After the obstacle movement is completed, the PID values are reset and the robot continues normal line following. The delay and timeout values were adjusted through repeated physical tests.

7. Simulation and Testing Results

At the beginning of the project, Tinkercad simulations were designed and performed to verify the circuit connections and identify potential errors before assembling the physical hardware.

After completing the individual component tests, a full Tinkercad simulation (Fig. 9) of the entire system was developed to verify that all components operated together as intended.

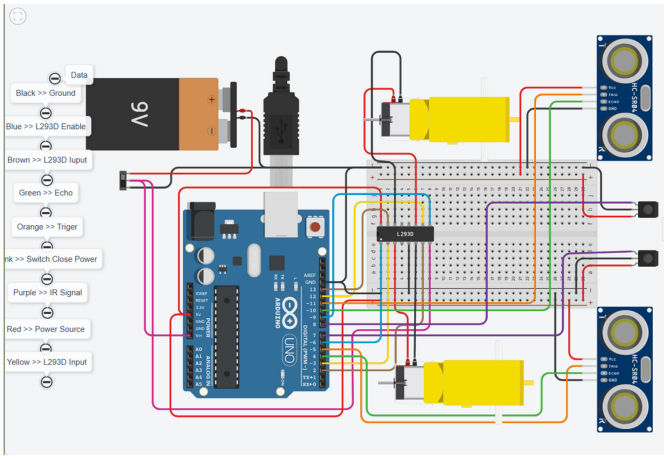

Fig. 9. System Wiring Diagram

7.1 System Verification Matrix

Table 5 summarises the verification results by listing the system requirements, verification methods, and test outcomes.

System Requirement	Verification Method	Pass/Fail
Follow predefined black track	Physical track test	PASS
Detect obstacle within 15 cm	Distance calibration in code	PASS
Execute obstacle bypass manoeuvre	Integrated track test	PASS
Execute second-obstacle 180° U-turn	Integrated track test	PASS
Battery runtime > 30 min	Endurance test	PASS

Table 4. System Verification Matrix

7.2 Performance Evaluation

After implementing the line-following and obstacle-avoidance functions, three experimental trials were conducted to evaluate the robot's performance. The results are summarized in Table 6. The robot successfully completed the line-following task in all trials without deviating from the line. In addition, the robot detected every obstacle during testing. Although one collision occurred, the robot successfully avoided the obstacle in two of the three trials. These results demonstrate that the robot provides reliable line-following performance and generally effective obstacle avoidance, while further improvements could increase the consistency of obstacle avoidance.

Line Following Test			
Trial	**Path Completed**	**Time (s)**	**Line Deviation**
1	Yes	47	No
2	Yes	40	No
3	Yes	33	No
Obstacle Avoidance Test			
Trial	**Obstacle Detected**	**Collision**	**Obstacle Avoided**
1	Yes	No	Yes
2	Yes	Yes	No
3	Yes	No	Yes

Table 5. Performance Evaluation Results

8. Challenges, Limitations and Future Improvements

8.1 Challenges and Solutions

During the development of the autonomous mobile robot, several challenges were encountered. The major issues and the corresponding solutions are summarised in Table 7.

Challenge	Description	Solution
Damaged Motor	The wiring of the right motor became disconnected, causing the motor to stop functioning.	Repaired and re-secured the motor wiring.
Damaged IR Sensor	The left IR sensor failed two weeks before the demonstration. The fault was identified when the robot could not perform left turns correctly.	Replaced the faulty IR sensor and recalibrated the system.
Line Deviation on Sharp Turns (Wiggling)	The robot deviated from the line during sharp turns because the steering response was unstable. The motors over-corrected left and right, causing the robot to drift off the line.	Implemented and tuned PID control by adjusting K_p and K_d. This eliminated wiggling and ensured smooth, accurate line-following during sharp turns.

Table 6. Challenges Encountered and Solutions

8.2 Current Limitations

Currently, the robot avoids obstacles by using predefined delay and timeout values. These parameters perform well in the current test environment but cannot guarantee successful obstacle avoidance under all conditions. Obstacles located at different positions or orientations may require further adjustment of the control parameters.

8.3 Future Improvements

A possible improvement is to install an additional ultrasonic sensor on the right side of the robot. By introducing a distance threshold instead of relying primarily on fixed delay values, the robot could adapt to obstacles at different positions and achieve more reliable obstacle avoidance.

9. Commits and Team Members Contributions (GitHub)

As shown in Fig. 10 and Table 8, the team contributions were tracked using Git commit history.

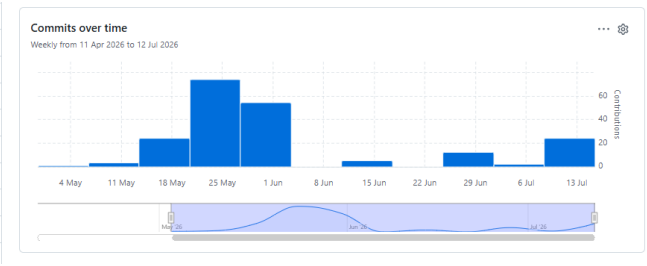

Fig. 10. Git Commit History

Team Member	Commits
Xintong Feng	54
Shamanta Islam	25
Elvis-Ledoux	66
Paula Cho	54
Total	199

Table 7. Team Contribution Summary

10. Conclusion

The autonomous vehicle prototype successfully demonstrated reliable line following and obstacle avoidance using low-cost hardware and PID-based control. Through systematic design, modular implementation, and iterative testing, the robot achieved stable navigation and consistent performance on the predefined track.

In addition to accurate line tracking, the robot was able to detect, avoid, and manoeuvre around obstacles, proving the effectiveness of the implemented control logic. Overall, this project represents a complete design process from initial concept to real-world implementation, combining both hardware and software development into a functional autonomous system.

11. Lessons Learned

- We learned the importance of teamwork and clear communication throughout the project.
- Testing the robot in real conditions helped us identify problems that were not obvious during initial testing and simulations.
- Solving hardware and software issues improved our understanding of embedded systems and the value of continuous testing and debugging.
- Using Git regularly made it easier to track progress, manage changes, and coordinate tasks within the team.
- We found that assigning tasks according to each team member's strengths improved our teamwork and helped us complete the project more efficiently.
- This project not only improved our technical skills but also increased our confidence in solving real engineering problems and working effectively as a team.

References

[1] Handson Technology, "L298N Dual H-Bridge Motor Driver User Guide," [Online]. Available: `https://www.handsontec.com/dataspecs/module/L298N%20Motor%20Driver.pdf`. [Accessed: Jul. 16, 2026].

[2] Arduino, "UNO R3," Arduino Documentation. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/. [Accessed: Jul. 16, 2026].

[3] J. Wilkie, M. Johnson, and R. Katebi, *Control Engineering: An Introductory Course*. Basingstoke, U.K.: Palgrave Macmillan, 2002.

[4] Shamanta14, "HSHL-Prototyping-E2-E4: Design and Construction of an Autonomous Vehicle That Can Follow a Line, Detect, and Avoid Obstacles," GitHub repository. [Online]. Available: https://github.com/Shamanta14/HSHL-Prototyping-E2-E4. [Accessed: Jul. 16, 2026].